Laporan Kerja Dev 2 (Server dan AI)

SpectraMeet MVP 1 - periode 14 sampai 19 Oktober 2026

Proyek	SpectraMeet MVP 1: mengubah rekaman meeting menjadi teks per pembicara, lalu membuat ringkasan.
Peran	Dev 2: membuat mesin pengolah audio dan AI. Dalam laporan ini mesin itu disebut **pipeline** (jalur pengolahan: rekaman masuk, hasil keluar).
Catatan tanggal	Pengerjaan nyata berlangsung 17-18 Oktober 2026 dan mengikuti urutan jadwal. Hasilnya dikelompokkan menurut tanggal rencana. Pekerjaan jadwal 20-21 Oktober (penanganan error dan lanjut dari tahap gagal) sudah ikut dikerjakan lebih awal.
Laptop uji	Intel Core i9-14900HX, RAM 64 GB, Windows 11. Ada kartu grafis NVIDIA RTX 4060, tetapi belum dipakai (lihat bagian GPU).

1. Ringkasan singkat

Pipeline sekarang sudah bisa bekerja dari awal sampai akhir di laptop. Ia menerima file rekaman, lalu menghasilkan **teks yang dibagi per pembicara**, ditambah **ringkasan, keputusan, tugas, dan pertanyaan terbuka**. Semuanya berjalan di laptop sendiri, tanpa server.

- Lima tahap pemrosesan sudah berjalan dan terbukti pada **rekaman nyata**, bukan data contoh.
- Rekaman 5 menit selesai sekitar **3 menit** di laptop ini, memakai memori sekitar **4 GB**.
- Jika satu tahap gagal, pekerjaan bisa **dilanjutkan dari tahap itu** tanpa mengulang dari awal.
- Pesan kesalahan dibuat jelas, misalnya "File audio rusak atau formatnya tidak dikenali".
- **55 tes otomatis** lulus.

Pekerjaan	Status	Keterangan
Pasang alat dan akun (FFmpeg, Hugging Face, WhisperX)	**Selesai**	Semua terpasang dan berjalan di Windows.
Uji tahap 2-4 pada 3 rekaman, laporan kecepatan	**Selesai**	Hasil di bagian 3.1.
Kerangka program, laporan progres, hasil dalam bentuk JSON	**Selesai**	Bagian 3.2 dan 3.3.
Tahap 5: ringkasan dengan Gemini (AI dari Google)	**Selesai**	Sudah diuji dengan Gemini sungguhan.
Penanganan error	**Sebagian**	Pesan jelas sudah ada. Catatan ke file (log) belum ada. "Memori habis" baru diuji lewat simulasi.
Lanjut dari tahap yang gagal	**Selesai**	Terbukti dengan model nyata.
Mempercepat di CPU	**Selesai**	Sudah dicoba. Tidak ada cara yang layak, pengaturan bawaan sudah terbaik.
Mempercepat dengan GPU	**Belum**	File pendukung sudah terunduh, belum dipasang dan diukur.
Pilihan ukuran model	**Sebagian**	Sementara memakai ukuran "small". Keputusan akhir menunggu penilaian kualitas oleh PM.
Uji 10 rekaman, rekaman panjang, setup dari nol, README pipeline	**Belum**	Direncanakan di minggu berikutnya.

2. Cara kerja pipeline

Pipeline bekerja dalam lima tahap berurutan. Setiap tahap melaporkan progresnya supaya halaman web bisa menampilkannya.

Tahap	Apa yang terjadi	Alat yang dipakai
1. Menyiapkan audio	Mengubah file audio apa pun menjadi satu format standar.	FFmpeg
2. Mendeteksi suara	Mencari bagian yang ada orang bicara, melewati bagian hening.	Pyannote (di dalam WhisperX)
3. Mengubah suara ke teks	Menuliskan ucapan menjadi teks, lalu mencocokkan waktu tiap kalimat.	Whisper (di dalam WhisperX)
4. Mengenali pembicara	Menentukan bagian mana diucapkan oleh siapa (Pembicara 0, 1, 2, dan seterusnya).	Pyannote
5. Menyusun kesimpulan	AI membuat ringkasan, keputusan, tugas, dan pertanyaan terbuka dari teksnya.	Gemini (Google)

Kamus istilah

Pipeline: jalur pengolahan. Rekaman masuk di satu ujung, hasil keluar di ujung lain.

FFmpeg: aplikasi gratis untuk mengubah format audio dan video.

Whisper / WhisperX: program AI yang mengubah suara menjadi teks (WhisperX adalah versi yang lebih lengkap).

Diarization (pengenalan pembicara): menentukan siapa yang berbicara di tiap bagian. Hasilnya berupa label seperti SPEAKER_00 dan SPEAKER_01. Label ini hanya penanda suara yang sama, bukan nama orang.

Alignment (pencocokan waktu): mencocokkan tiap kata dengan detik di rekaman, sehingga teks bisa dipecah per kalimat dengan waktu yang tepat.

LLM / Gemini: AI yang bisa membaca dan menulis teks. Gemini adalah buatan Google. Di sini dipakai untuk membuat ringkasan.

Token Hugging Face dan lisensi pyannote: model pengenal pembicara disimpan di situs Hugging Face dan harus disetujui dulu lisensinya. Token adalah "kunci" akun untuk mengunduhnya.

CPU dan GPU: CPU adalah otak utama laptop. GPU (kartu grafis) lebih cepat untuk hitungan AI tertentu.

JSON: format file teks yang rapi dan mudah dibaca program. Hasil pipeline disimpan dalam format ini.

Checkpoint dan resume: checkpoint adalah hasil sementara yang disimpan tiap tahap. Resume berarti melanjutkan dari tahap yang gagal memakai hasil sementara itu.

Halusinasi AI: AI menulis sesuatu yang tidak ada di sumbernya, misalnya mengulang kata yang tidak diucapkan atau mengarang nama.

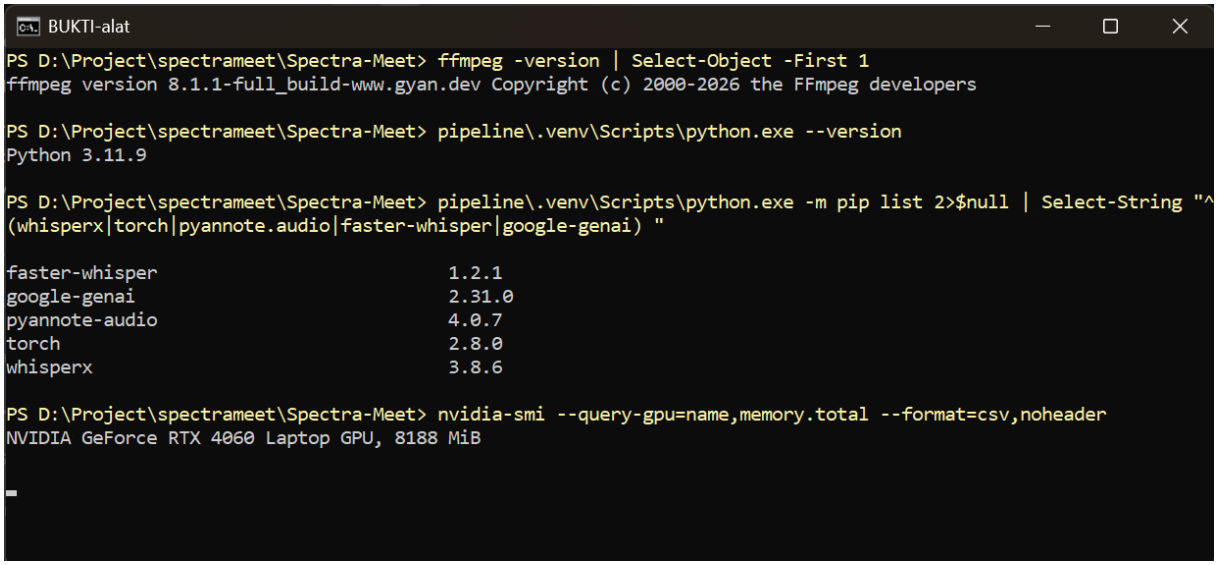
3. Pekerjaan menurut jadwal

Semua gambar di bawah adalah **tangkapan layar asli** dari laptop saat perintah dijalankan. Layar hitam adalah jendela perintah (PowerShell). Tulisan kuning adalah perintah yang dijalankan, tulisan putih adalah hasilnya. Tidak ada gambar yang dibuat atau diedit, hanya dipangkas bagian tepinya.

3.1 Rabu 14 Oktober: pasang alat dan uji awal

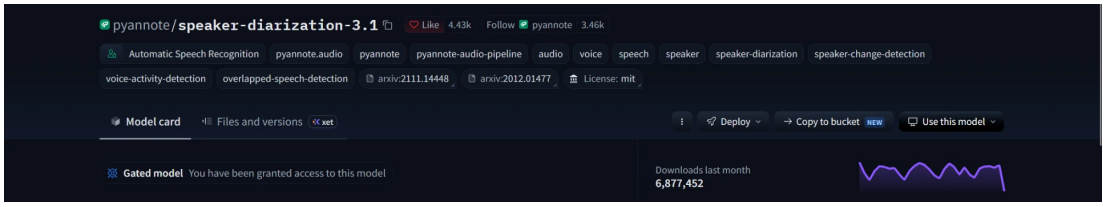
Target: memasang FFmpeg, akun Hugging Face, lisensi pyannote, dan lingkungan pipeline, lalu menjalankan tahap 2-4 pada rekaman dan mencatat waktu, memori, serta masalah khas Windows.

Hasil: semua alat terpasang. Gambar berikut menunjukkan versi alat yang terpasang. Perhatikan bahwa torch (mesin hitung AI) masih versi 2.8.0 biasa, artinya berjalan di CPU, sedangkan kartu grafis RTX 4060 terdeteksi tetapi belum dipakai.

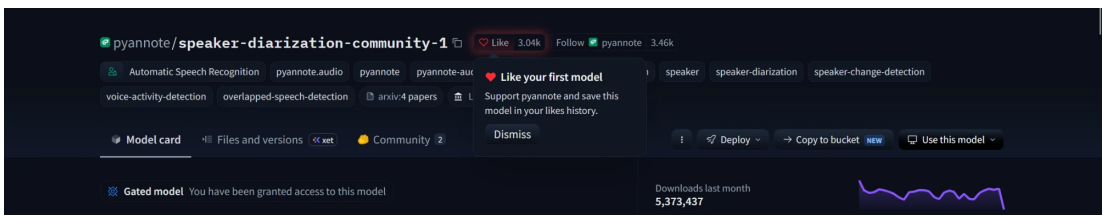


Gambar 1. Versi alat yang terpasang: FFmpeg, Python, paket AI (WhisperX, torch, pyannote, faster-whisper, google-genai), dan kartu grafis.

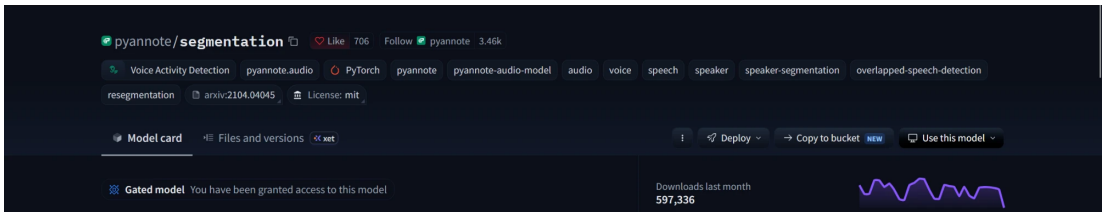
Model pengenal pembicara memerlukan persetujuan lisensi di Hugging Face. Tiga halaman model sudah menunjukkan "You have been granted access to this model" (akses sudah diberikan). Awalnya diperkirakan model *speaker-diarization-3.1* yang dipakai, tetapi setelah kode WhisperX diperiksa ternyata yang dipakai adalah *community-1* dan *segmentation*, sehingga ketiganya disetujui.



Gambar 2a. Halaman model speaker-diarization-3.1 di Hugging Face: akses sudah diberikan.



Gambar 2b. Halaman model speaker-diarization-community-1 (yang benar-benar dipakai WhisperX): akses sudah diberikan.



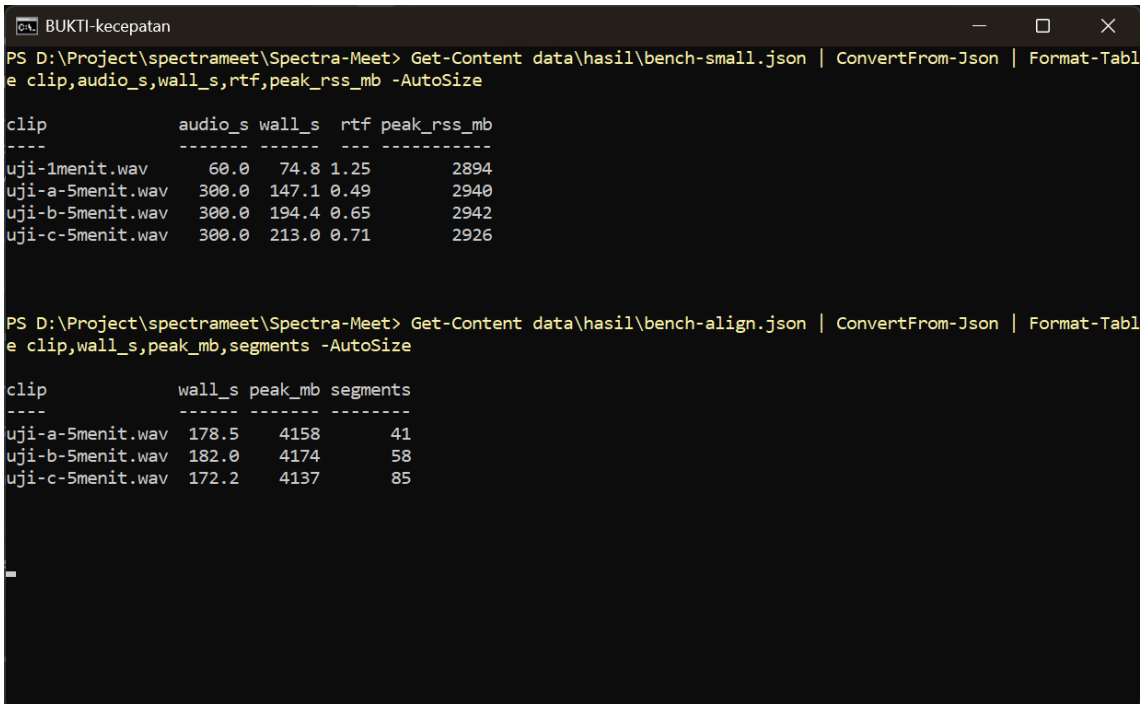
Gambar 2c. Halaman model segmentation (untuk mendeteksi suara): akses sudah diberikan.

Rekaman uji: dipotong dari satu rekaman asli berdurasi 4 jam 12 menit, menjadi satu klip 1 menit dan tiga klip 5 menit (A, B, C). File aslinya tidak diubah.

Hasil uji kecepatan (tahap 2-4, ukuran model "small", belum memakai pencocokan waktu):

Klip	Lama rekaman	Waktu proses	Memori puncak	Pembicara terdeteksi
1 menit	60 detik	74,8 detik	2,9 GB	1
A (5 menit)	300 detik	147,1 detik	2,9 GB	1
B (5 menit)	300 detik	194,4 detik	2,9 GB	2
C (5 menit)	300 detik	213,0 detik	2,9 GB	3

Klip 1 menit terasa lebih lambat per detik rekaman karena ada waktu tetap untuk memuat model.



Gambar 3. Angka mentah hasil pengukuran. Tabel atas: tanpa pencocokan waktu (kolom wall_s = waktu proses dalam detik, rtf = perbandingan waktu proses dengan lama rekaman, peak_rss_mb = memori puncak dalam MB). Tabel bawah: setelah pencocokan waktu ditambahkan (bagian 3.2).

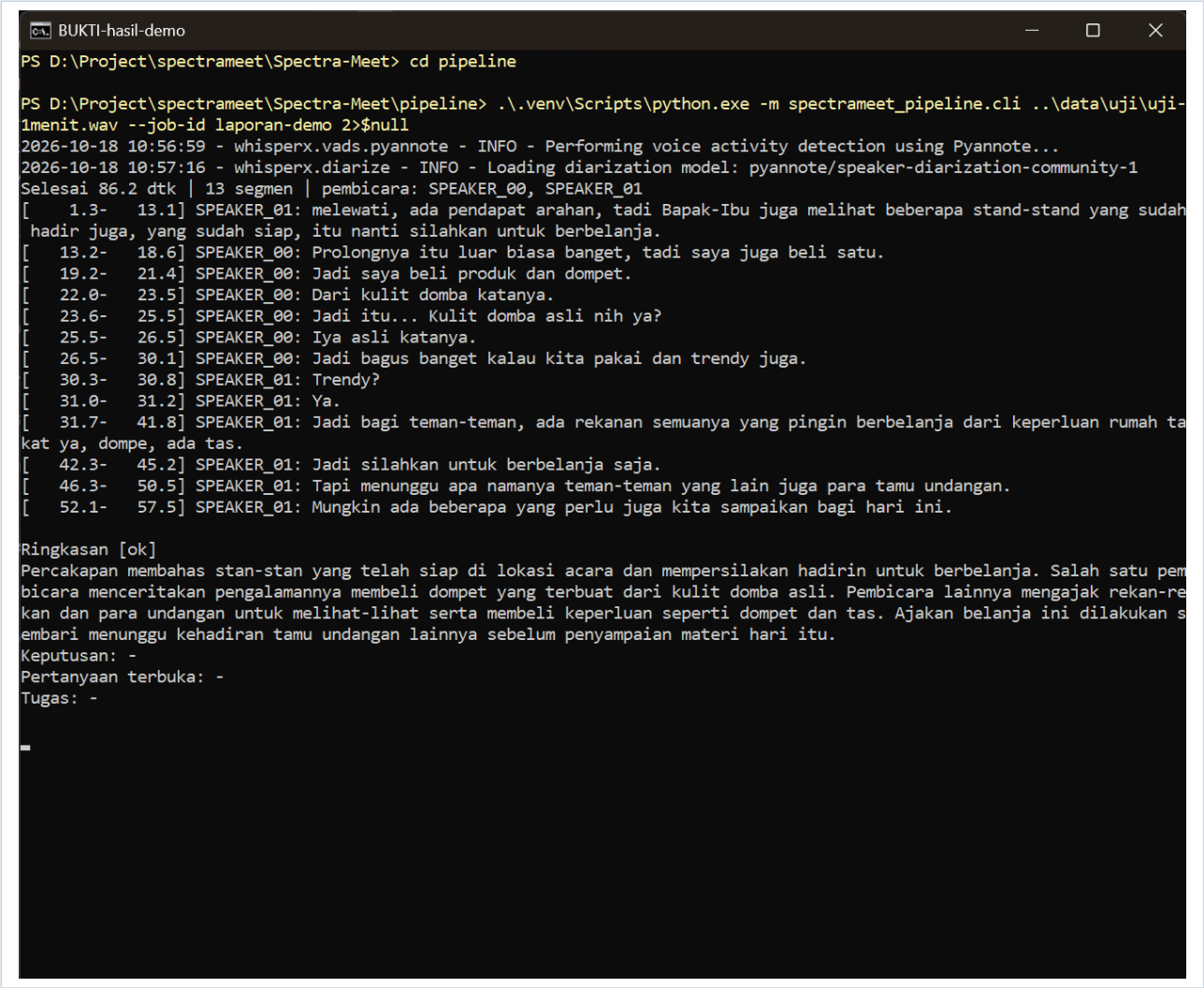
Masalah khas Windows yang ditemukan:

- Unduhan model sempat macet di 0 byte karena batas waktu bawaan hanya 10 detik. Diperbaiki dengan memperpanjang batas waktu dan mematikan klien unduhan yang bermasalah (diatur otomatis oleh program).
- Muncul banyak peringatan tentang *torchcodec* dan *symlink*. Keduanya hanya peringatan dan tidak menghentikan proses.

3.2 Kamis 15 Oktober: transkrip berlabel pembicara dan kerangka program

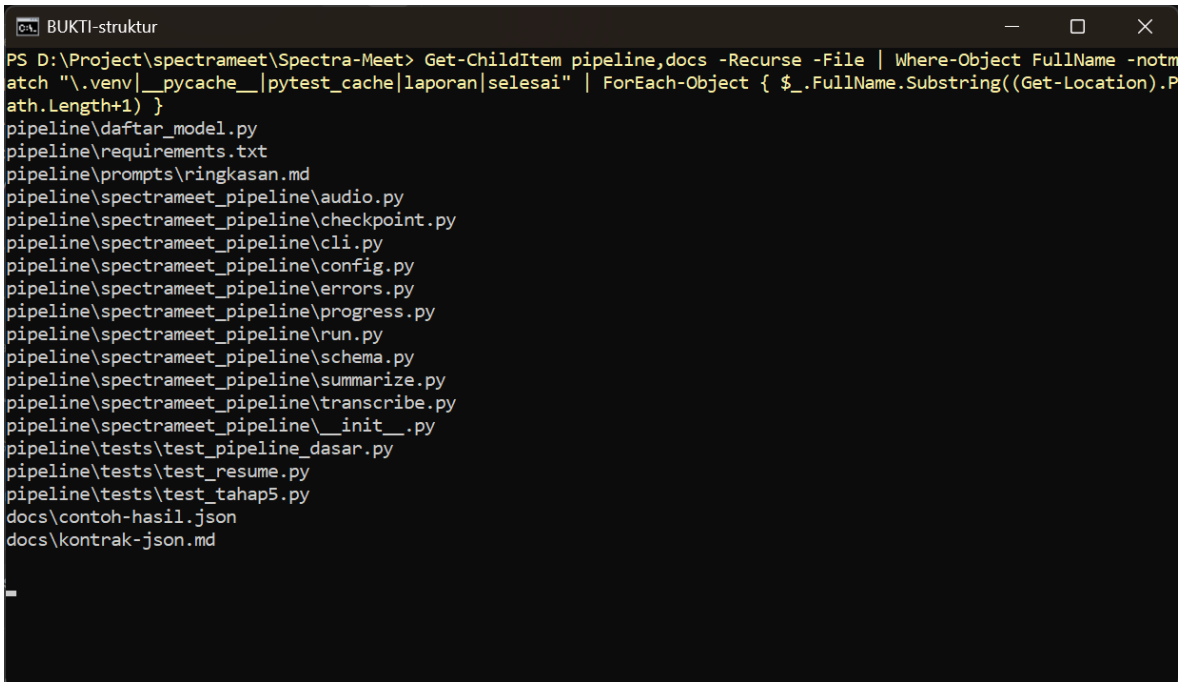
Target: demo transkrip berlabel pembicara di terminal, laporan kecepatan, dan kerangka program *process()* (pintu masuk tunggal pipeline).

Hasil: pipeline dijalankan dari terminal pada klip 1 menit. Hasilnya 13 kalimat dengan waktu mulai dan selesai, dibagi ke Pembicara 0 dan Pembicara 1, lalu ringkasan dari Gemini. Tanpa pencocokan waktu, klip yang sama hanya menghasilkan 3 potongan panjang dan 1 pembicara, jadi tahap ini penting.



Gambar 4. Pipeline lengkap pada klip 1 menit: 13 segmen, 2 pembicara, selesai 86,2 detik, dan ringkasan berstatus [ok]. Teks belum sempurna (ada salah dengar seperti "melewati, ada pendapat arahan"). Menilai ketepatan teks adalah tugas PM.

Program disusun menjadi beberapa file kecil, masing-masing punya satu tugas (audio, transkripsi, progres, ringkasan, dan seterusnya). Ada juga file tes dan prompt (instruksi untuk AI) yang bisa diubah tanpa menyentuh kode.

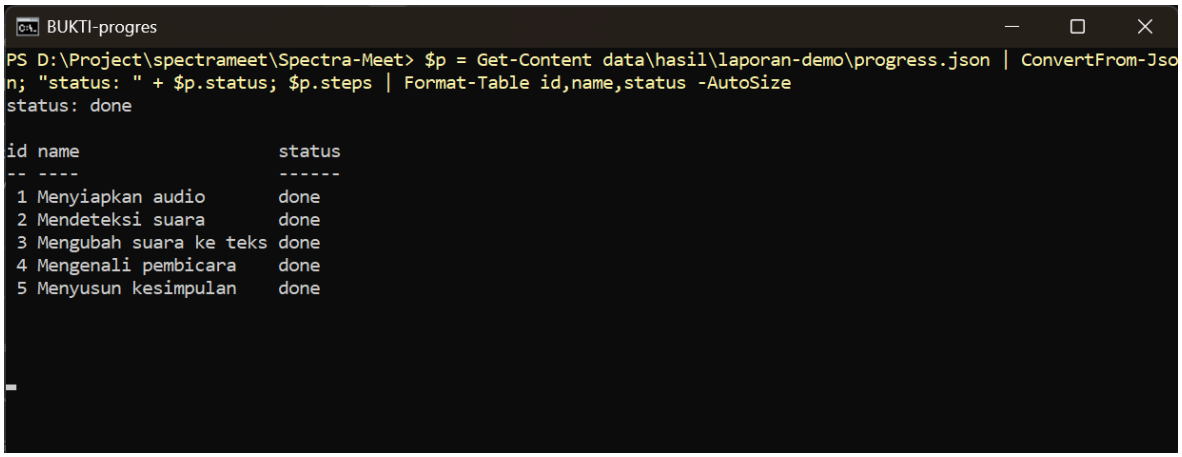


Gambar 5. Daftar file hasil kerja Dev 2 di folder pipeline dan docs.

3.3 Jumat 16 Oktober: laporan progres, hasil JSON, cek kelayakan

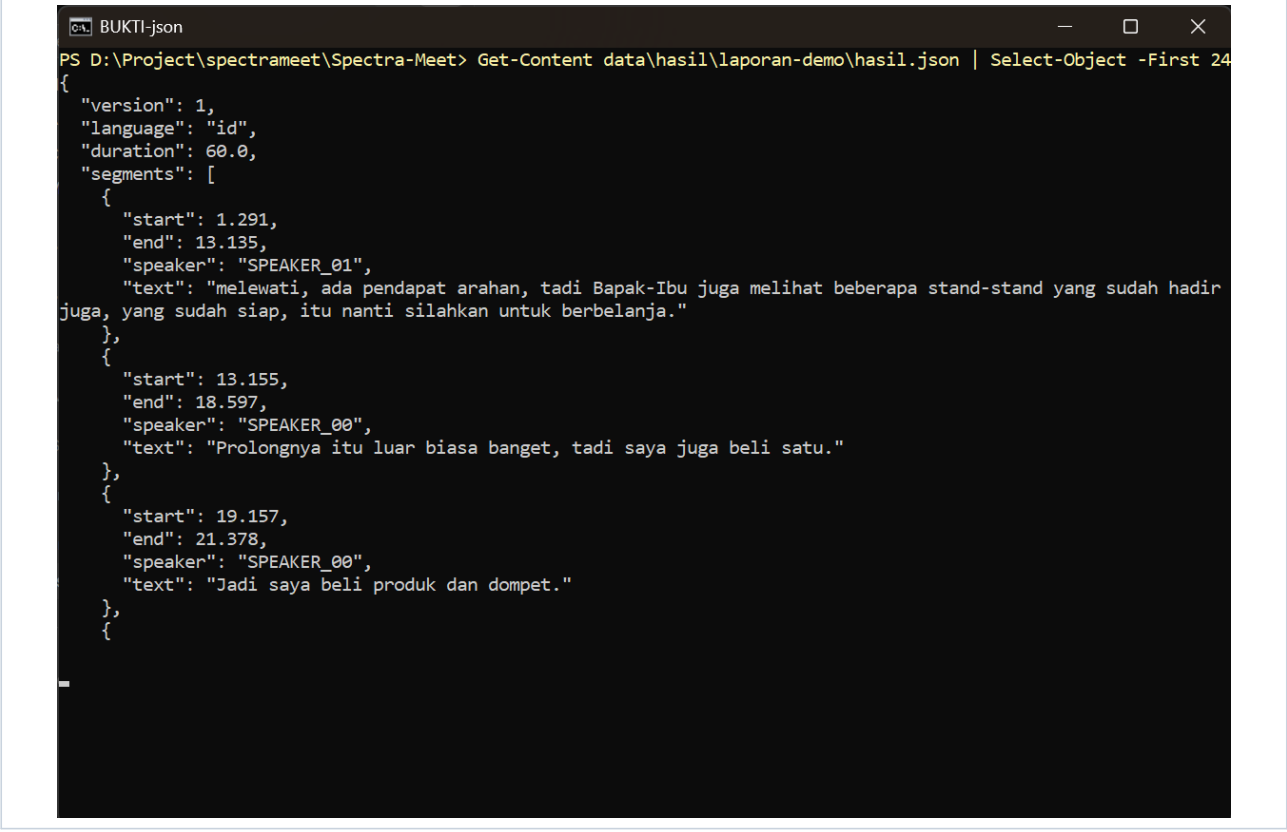
Target: laporan progres tahap 1-5 dan hasil dalam format JSON sesuai kontrak, lalu cek kelayakan: apakah tahap 2-4 terbukti jalan di laptop.

Hasil: setiap tahap menulis keadaannya ke file *progress.json* (belum mulai, berjalan, selesai, dilewati, atau gagal). Halaman web akan membaca file ini untuk menampilkan lima langkah. File ditulis dengan cara aman supaya tidak pernah terbaca setengah jadi.



Gambar 6. Keadaan lima tahap setelah pipeline selesai: semuanya done (selesai).

Hasil akhir disimpan di file *hasil.json*: bahasa, durasi, daftar kalimat (waktu mulai, waktu selesai, pembicara, teks), lalu ringkasan, keputusan, tugas, dan pertanyaan terbuka. Bentuk file ini sudah ditulis di *docs/kontrak-json.md* sebagai **draf**, lengkap dengan contoh di *docs/contoh-hasil.json*, untuk disetujui PM dan Dev 1.



Gambar 7. Awal file hasil.json yang dihasilkan pipeline (kalimat berisi waktu, pembicara, dan teks).

Kesimpulan cek kelayakan 16 Oktober

Layak dilanjutkan. Tahap 2-4 terbukti berjalan di laptop Windows tanpa kartu grafis. Rekaman 5 menit selesai sekitar 3 menit dan memakai memori sekitar 4 GB. Rencana tidak perlu diubah.

Perlu diingat: angka ini baru dari satu rekaman asli dan satu laptop. Rekaman panjang dan laptop Dev 1 belum diuji.

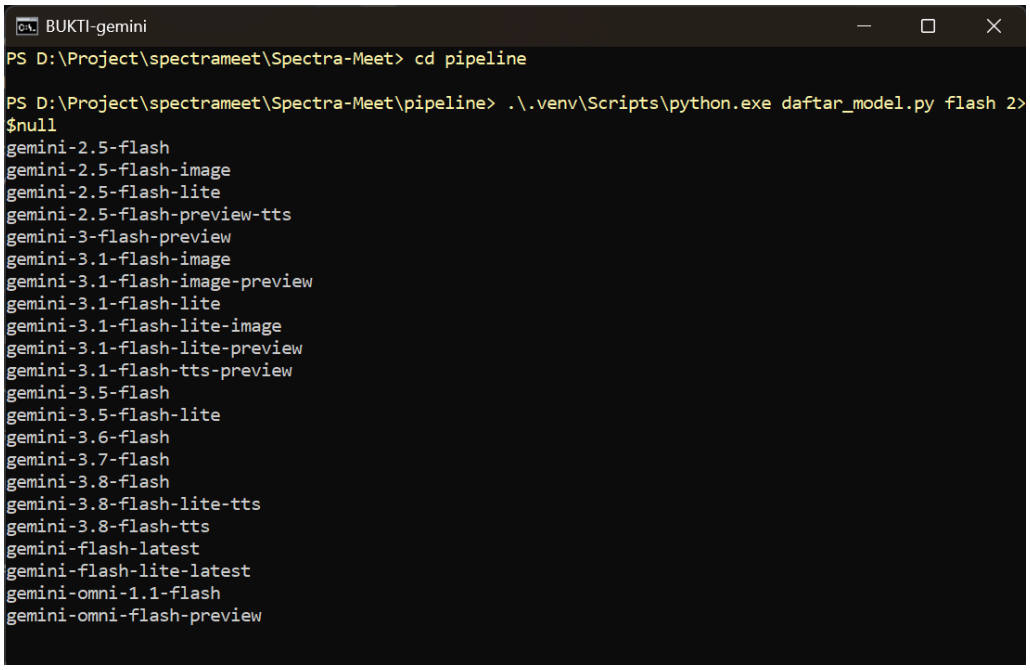
3.4 Senin 19 Oktober: tahap 5, ringkasan dengan Gemini

Target: satu panggilan AI untuk membuat ringkasan, hasilnya diperiksa bentuknya, dan ada cadangan jika jawaban AI rusak.

Hasil: tahap 5 selesai dan sudah diuji dengan Gemini sungguhan (kunci API milik sendiri). Cara kerjanya:

- Teks dikirim ke Gemini beserta instruksi. Jawaban harus berbentuk JSON dengan empat bagian: ringkasan, keputusan, tugas, pertanyaan terbuka.
- Jawaban diperiksa bentuknya. Jika rusak, program meminta sekali lagi dengan catatan perbaikan.
- Jika Gemini sedang sibuk (galat 503), program menunggu lalu mencoba lagi sampai 3 kali.
- **Jika tahap 5 tetap gagal, teks per pembicara tetap tersimpan.** Pengguna tidak kehilangan hasil tahap 1-4.

Nama model yang bisa dipakai dicek langsung ke akun, bukan dari daftar di internet, karena daftar model sering berubah.



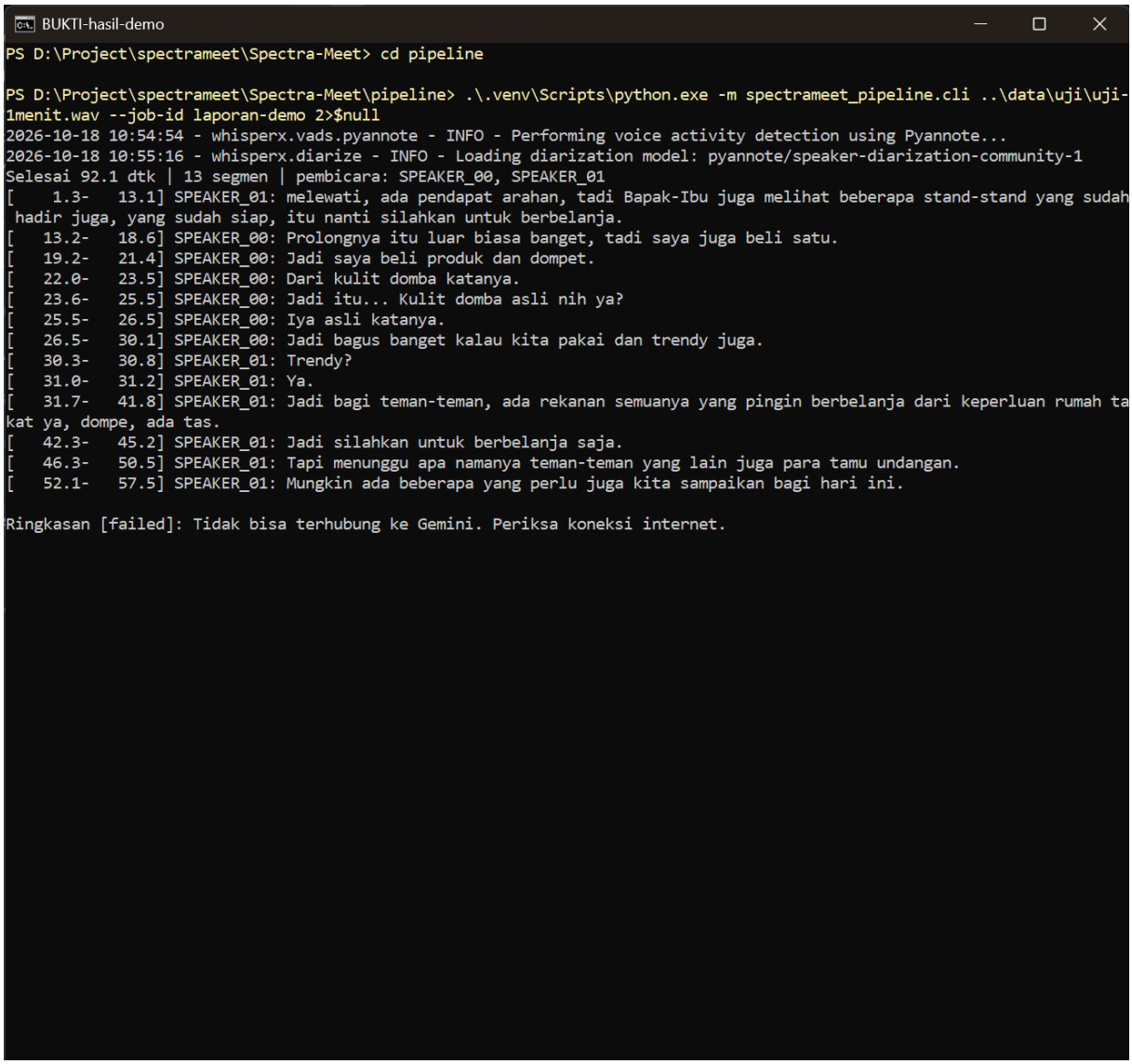
Gambar 8. Daftar model Gemini yang tersedia untuk kunci ini (hanya yang bernama "flash"). Empat model yang tadinya dikira gratis (gemini-3.8-flash, 3.5-flash-lite, 3.6-flash, 3.1-flash-lite) memang ada di daftar. Apakah akun ini gratis atau berbayar belum diketahui.

Perbandingan dua model pada teks klip C (85 kalimat, 3 pembicara):

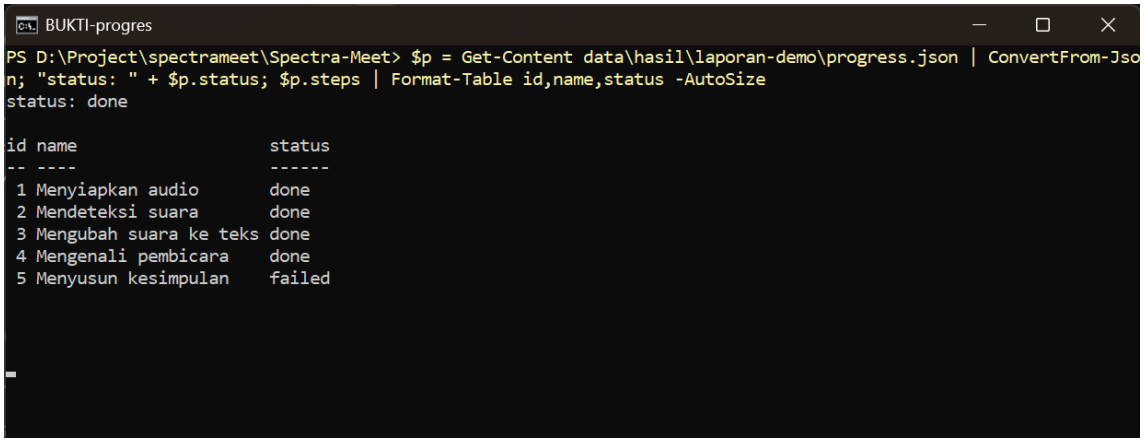
Model	Waktu	Temuan
gemini-3.1-flash-lite	6,0 detik	**Mengarang pemilik tugas**: menulis "Peserta" dan "Panitia", padahal kedua kata itu tidak ada di teks. Ini melanggar aturan di instruksi.
gemini-3.8-flash	6,7 detik	Mengosongkan pemilik tugas (null) karena tidak disebut. Sesuai aturan.

Rekomendasi sementara: gemini-3.8-flash. Ini baru satu contoh, jadi bukan kesimpulan akhir. Untuk PM: instruksi (prompt) perlu menegaskan bahwa peran umum seperti "panitia" atau "peserta" bukan nama pemilik tugas.

Pada percobaan pertama demo, koneksi internet ke Gemini sedang putus. Hasilnya menunjukkan cadangan bekerja: teks dan pembicara tetap tersimpan, hanya ringkasan yang ditandai gagal dengan alasan jelas.



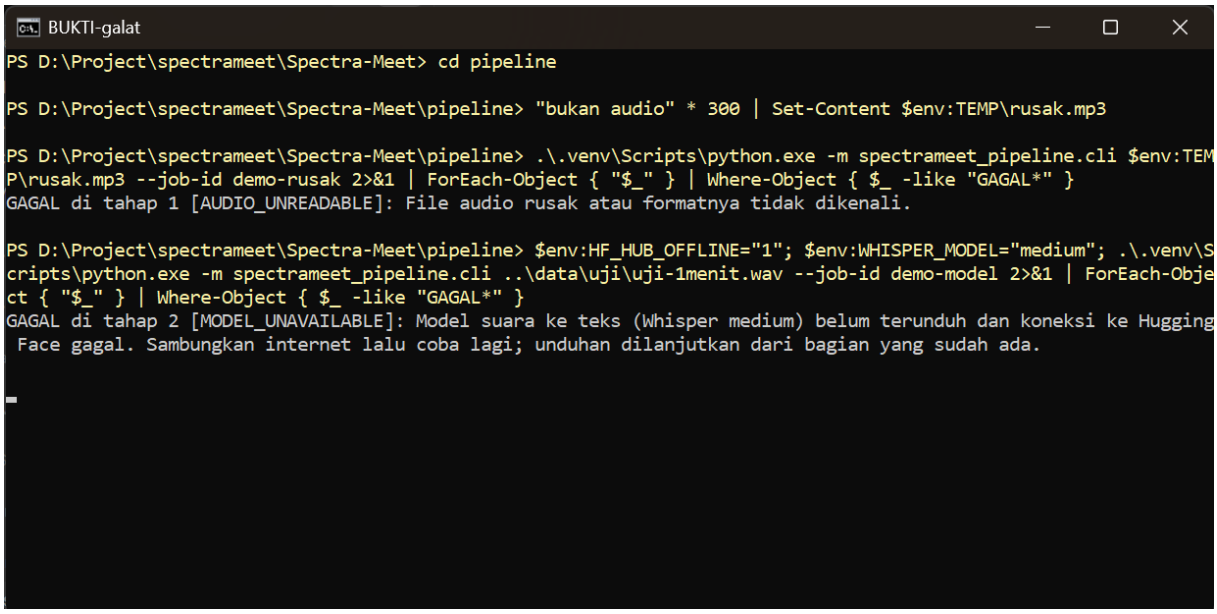
Gambar 9. Percobaan saat koneksi ke Gemini putus: 13 segmen dan 2 pembicara tetap tersimpan, ringkasan berstatus [failed] dengan pesan "Tidak bisa terhubung ke Gemini. Periksa koneksi internet."



Gambar 10. Keadaan tahap pada percobaan itu: tahap 1-4 done, hanya tahap 5 yang failed. Status pekerjaan tetap done.

3.5 Dikerjakan lebih awal: penanganan error, lanjut dari tahap gagal, dan percepatan

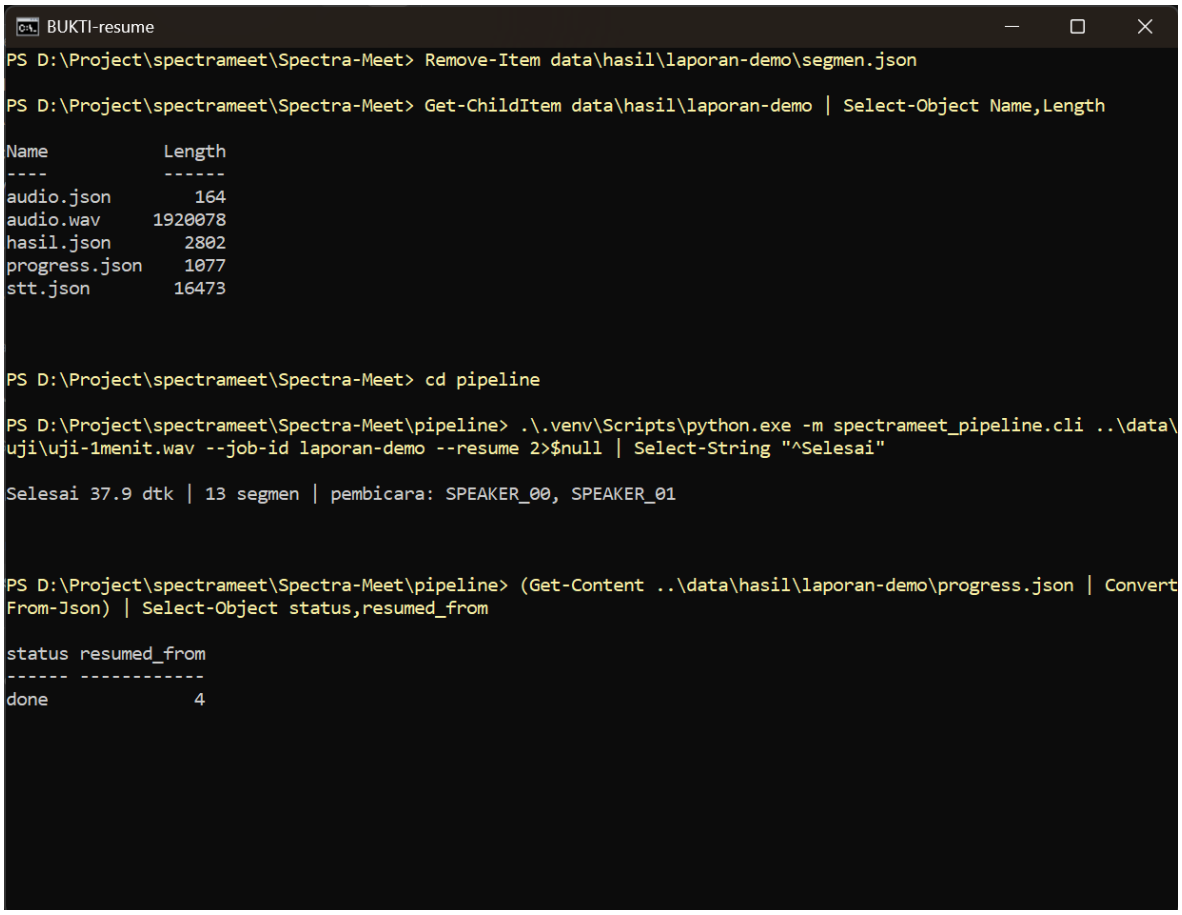
Penanganan error. Dua kegagalan nyata diuji: file audio rusak dan model yang belum terunduh saat tidak ada internet. Pesannya menyebut tahap, penyebab, dan langkah perbaikan.



Gambar 11. Dua kegagalan yang disengaja. Atas: file rusak, gagal di tahap 1 (AUDIO_UNREADABLE). Bawah: model Whisper "medium" belum terunduh dan internet dimatikan, gagal di tahap 2 (MODEL_UNAVAILABLE) dengan saran mencoba lagi.

Daftar kode kesalahan yang dikenal pipeline: file audio tidak ada, FFmpeg belum terpasang, file rusak, token Hugging Face kosong, model tidak bisa diakses, memori habis, dibatalkan, serta beberapa kode khusus untuk tahap 5 (misalnya kunci ditolak, model tidak ditemukan, kuota habis, Gemini sibuk, tidak terhubung, jawaban rusak).

Lanjut dari tahap yang gagal. Tiap tahap menyimpan hasilnya di folder pekerjaan. Jika satu tahap gagal, perintah yang sama dengan tambahan *--resume* melewati tahap yang sudah selesai. Hasil sementara hanya dipakai kembali jika rekaman dan pengaturannya sama, supaya tidak salah pakai.



Gambar 12. File hasil sementara segmen.json dihapus untuk meniru kegagalan di tahap 4. Dengan --resume pipeline selesai dalam 37,9 detik (dari awal tadi 86,2 detik) dan progress.json mencatat resumed_from = 4 (mulai dihitung ulang dari tahap 4).

Pada klip 5 menit penghematannya lebih besar: dari awal sekitar 190 detik, lanjut dari tahap 4 sekitar 104 detik, dan mengulang ringkasan saja hanya sekitar 8 detik.

Tes otomatis. 55 tes memeriksa konversi audio, laporan progres, bentuk hasil, tahap 5 (dengan AI palsu agar tidak perlu internet), dan semua skenario lanjut dari tahap gagal.



Gambar 13. Seluruh tes otomatis dijalankan: 55 passed (lulus) dalam 3 detik.

Percobaan mempercepat di CPU. Tahap pengenalan pembicara memakan sekitar 57% waktu total, jadi dicoba beberapa pengaturan pada klip C. Kesimpulannya: pengaturan bawaan sudah paling baik.

Percobaan	Hasil	Keputusan
Memperbesar langkah analisis pengenal pembicara	2 sampai 5 kali lebih cepat, tetapi 1 pembicara hilang (3 menjadi 2) dan kecocokan label turun ke 41-66%	Ditolak, merusak ketepatan
Mengubah jumlah pekerja CPU untuk pengenal pembicara (24, 16, 12, 8)	Tetap 24 yang tercepat (61 detik, lainnya 66-72 detik)	Tetap bawaan
Mengubah ukuran paket data (8, 32, 64)	32 dan 64 sama cepat, 8 lebih lambat (80 detik)	Tetap bawaan
Menambah pekerja Whisper menjadi 8	Sedikit lebih lambat (36,4 dibanding 32,7 detik)	Tetap bawaan (4)
Menambah pekerja Whisper menjadi 16	Program mati mendadak (2 dari 2 percobaan)	Dihindari
Menjalankan transkripsi dan pengenal pembicara bersamaan	Lebih lambat: keduanya berebut CPU. Whisper 69 detik (sendirian 33) dan pengenal pembicara 107 detik (sendirian 61)	Ditolak

Status GPU (kartu grafis)

Satu-satunya jalan nyata untuk mempercepat adalah GPU, karena bagian terlama (mengenali pembicara) cocok dihitung di GPU. File torch versi GPU berukuran 2,9 GB dan sempat sangat lambat diunduh. **File itu sekarang sudah selesai terunduh, tetapi belum dipasang dan belum diukur.** Belum ada angka, jadi belum ada klaim percepatan.

4. Masalah yang ditemukan dan solusinya

Masalah	Apa artinya	Solusi
Unduhan model macet di 0 byte	Batas waktu bawaan 10 detik terlalu pendek untuk internet lambat.	Batas waktu dinaikkan dan klien unduhan bermasalah dimatikan, diatur otomatis.
Proses tidak mau berhenti setelah selesai	Pustaka AI diam-diam mengunduh file tambahan 1,2 GB di latar belakang.	Fitur itu dimatikan lewat pengaturan resmi. Proses kini selesai dan keluar sendiri.
Gemini menjawab "sibuk" (503)	Gangguan sementara di sisi Google.	Dicoba ulang otomatis sampai 3 kali. Jika tetap gagal, teks tetap tersimpan.
Model murah mengarang pemilik tugas	AI menulis "Peserta" dan "Panitia" yang tidak ada di teks.	Memakai model gemini-3.8-flash. Prompt perlu dipertegas oleh PM.
Satu segmen berisi "iya" berulang sekitar 90 kali (klip B, detik 54-74)	Kemungkinan halusinasi Whisper pada bagian hening atau berisik. Belum didengarkan manusia.	Perlu dicek PM. Opsi perbaikan: atur deteksi suara atau parameter Whisper.
Model terlalu besar untuk rekaman panjang?	Belum diketahui. Rekaman 4 jam belum pernah diproses utuh.	Akan diuji di langkah berikutnya.

5. Yang belum selesai dan risikonya

- **Uji 10 rekaman** (target: minimal 8 dari 10 selesai tanpa error) belum dilakukan. Semua klip uji berasal dari satu rekaman.
- **Rekaman panjang** belum diuji. File 4 jam 12 menit belum diproses utuh. Waktu untuk 1 jam baru perkiraan (sekitar 35-40 menit).
- **Ketepatan teks dan label pembicara** belum dinilai manusia. Angka yang ada baru kecepatan dan jumlah pembicara. Ini tugas PM.
- **Gabungan dengan API dan web** belum bisa diuji karena itu bagian Dev 1. Masalah jalur file, penulisan karakter, dan proses di Windows saat dipanggil API baru bisa dicari setelah itu.

- **Catatan kejadian (log) ke file** belum ada. Saat ini alasan kegagalan hanya ada di progress.json.
- **README bagian pipeline, setup.ps1, dan run-dev.ps1** serta uji setup dari nol di laptop lain belum dibuat.
- **Kontrak JSON masih draf.** Nama field berbahasa Inggris dan beberapa usulan (misalnya tombol Batalkan) perlu disetujui PM dan Dev 1.
- **Hasil ukur baru dari satu laptop.** Laptop Dev 1 belum diuji.

6. Langkah berikutnya

1. Memasang torch versi GPU di lingkungan terpisah, lalu mengukur kecepatan pengenalan pembicara di GPU dan memeriksa apakah label pembicaranya sama.

2. Memproses rekaman penuh 4 jam untuk mengetahui batas waktu dan memori, lalu menyiapkan 10 klip beragam untuk uji "8 dari 10".

3. Menulis README bagian pipeline, setup.ps1, dan run-dev.ps1, lalu menguji setup dari nol.

4. Menambah catatan kejadian (log) ke file per pekerjaan.

5. Membahas kontrak JSON dan prompt ringkasan dengan PM dan Dev 1.

Lampiran: file hasil kerja

File	Fungsi
pipeline/spectrameet_pipeline/run.py	Pintu masuk tunggal *process()*: menjalankan tahap 1-5, mengatur lanjut dari tahap gagal, mengubah error menjadi pesan jelas.
pipeline/spectrameet_pipeline/audio.py	Tahap 1: mengubah audio ke format standar dengan FFmpeg.
pipeline/spectrameet_pipeline/transcribe.py	Tahap 2-4: deteksi suara, suara ke teks, pengenalan pembicara (WhisperX).
pipeline/spectrameet_pipeline/summarize.py	Tahap 5: ringkasan dengan Gemini, pemeriksaan jawaban, coba ulang.
pipeline/spectrameet_pipeline/progress.py	Menulis laporan progres ke progress.json.
pipeline/spectrameet_pipeline/checkpoint.py	Menyimpan dan memakai kembali hasil antar tahap.
pipeline/spectrameet_pipeline/schema.py, errors.py, config.py, cli.py	Bentuk data, daftar kode error, pembacaan pengaturan .env, dan perintah terminal.
pipeline/prompts/ringkasan.md	Instruksi untuk AI (draf awal dari Dev 2, PM menulis versi sebenarnya).
pipeline/daftar_model.py	Menampilkan model Gemini yang bisa dipakai oleh kunci di .env.
pipeline/tests/	55 tes otomatis.
docs/kontrak-json.md, docs/contoh-hasil.json	Draf kontrak bentuk hasil dan contoh untuk Dev 1.
docs/laporan-kecepatan-dev2.md	Laporan kecepatan rinci (bahasa teknis) sebagai pelengkap laporan ini.